IoT-Based Real-Time Foreign Particle Detection System for Tea Manufacturing Using Deep Learning

Dr. Bhagya Nathalie Silva
Faculty of Computing
Sri Lanka Institute of Information Technology (SLIIT)
nathali.s@sliit.lk

Liyanage S.N
Faculty of Computing
Sri Lanka Institute of Information Technology (SLIIT)
sachithnimaliyanage@gmail.com

De Silva P.S
Faculty of Computing
Sri Lanka Institute of Information Technology (SLIIT)
semaldesilva2002@gmail.com

Shavinda G.M.V
Faculty of Computing
Sri Lanka Institute of Information Technology (SLIIT)
vidulashavinda12@gmail.com

Sooriyaarachchi N.D
Faculty of Computing
Sri Lanka Institute of Information Technology (SLIIT)
navindu.ds01@gmail.com

***Abstract*—An ongoing problem in the tea manufacturing industry is the purity of the end product especially at the final packaging stage, where foreign materials such as threads, insect fragments, plastic pieces, stones can contaminate the product. This paper describes an IoT-based detection system built to catch these contaminants in real time, using a camera mounted above a conveyor belt and a deep learning model running on a local edge machine. When the model spots a foreign particle, it sends a signal over Wi-Fi to an ESP32 microcontroller, which stops the belt through a stepper motor so the object can be removed manually. Three most advanced models, namely Faster R-CNN, YOLOv11, and YOLOv8n, were trained on a specially created dataset based on the real production setting and compared relatively. YOLOv8n had the best mean average precision (mAP@0.5) of 91.5 at an inference speed of approximately 12 milliseconds per frame, which is the best candidate to run in real-time. The proposed system has great industrial applicability and provides a cost-effective and practical solution to automated quality control in tea processing.**

I. INTRODUCTION

A. Background

Tea is one of the most widely consumed beverages in the world, with global production exceeding six million metric tonnes annually [1]. Sri Lanka, India, China, and Kenya are among the largest producers, with Ceylon tea commanding particular prestige in premium export markets. In these segments, consumer expectations and regulatory standards are stringent: contamination of the final product can result in recalls, regulatory penalties, and lasting brand damage. Ensuring the physical purity of tea at the point of packaging is therefore a critical priority for manufacturers at every scale. Conventional quality control at the production line level relies predominantly on visual inspection by human operators. This approach is limited by operator fatigue during long shifts, naturally variable attention levels, inconsistent factory lighting, and the high throughput speeds of modern conveyor systems. Taken together, these factors make reliable manual detection of small foreign particles practically infeasible. Automated machine vision systems that can operate continuously and trigger an immediate mechanical response upon detection are therefore a compelling alternative.

B. Problem Statement

Foreign particle contamination in processed tea arises from multiple sources: cotton and synthetic threads shed by packaging equipment, insect fragments such as gecko body parts, small plastic shards from machinery wear, stone chips carried over during raw material handling, and miscellaneous environmental debris. Many of these objects are small, irregularly shaped, and share color or texture characteristics with natural tea particles, making them difficult to isolate visually or algorithmically. A system capable of detecting such objects at real-time conveyor speeds and automatically stopping the belt for manual removal would directly address a gap that manual inspection cannot reliably fill.

C. Research Objectives

This research aims to:

- Design and implement a conveyor-integrated camera and edge computing pipeline for real-time video processing.
- Construct a custom annotated image dataset of tea conveyor frames with realistic foreign particle contaminants.
- Train and comparatively evaluate three object detection models - Faster R-CNN, YOLOv11, and YOLOv8n on this dataset using standard detection metrics.
- Deploy the best-performing model within a Streamlit monitoring application that communicates with an ESP32 microcontroller over TCP to halt the conveyor belt on detection.
- Evaluate real-world system behavior and identify key limitations and future research directions.

II. LITERATURE REVIEW

A. Food Contamination Detection

Machine vision-based food inspection has evolved considerably over the past two decades. Early systems applied color thresholding and morphological image processing to detect surface defects with limited robustness to environmental variation [2]. Deep learning approaches have substantially raised the performance ceiling: CNNs have been shown to outperform traditional feature engineering in grain contamination detection, achieving rates above 93 under noisy imaging conditions [3]. In the tea domain specifically, hyperspectral and near-infrared imaging have been explored for quality grading [4], but their cost and complexity restrict adoption to large-scale operations and they do not generalize naturally to physical particle contamination.

B. Object Detection Architectures

The Faster R-CNN architecture [5][6] established the twostage detection paradigm, combining a Region Proposal Network with a classification head to achieve high localization precision. Its primary drawback for industrial deployment is inference latency: the two-stage pipeline is computationally expensive and difficult to run at real-time frame rates on modest hardware. The YOLO family [7] addressed this by unifying localization and classification into a single feedforward pass over a spatial grid, achieving dramatically lower inference times. Ultralytics YOLOv8 [8] introduced further architectural refinements, including an improved cross-stage partial bottleneck and detection head. The nano variant (YOLOv8n) minimizes parameter count to approximately 3.2 million while retaining the core architectural benefits, making it well-suited to edge deployment. YOLOv11 [9] followed with additional multi-scale feature fusion improvements, albeit at a larger model size.

C. IoT-Integrated Inspection Systems

Practical deployment of machine vision in manufacturing requires integration with physical actuation systems. Kumar and Patel [10] demonstrated ESP32-based actuator control coupled with image classification for light manufacturing lines, with end-to-end latencies below 200 milliseconds. Fernandez et al. [11] deployed YOLO-based detection integrated with PLC control in a bottling plant, reporting false positive rates below 2%. These works confirm the viability of real-time closed-loop vision-and-actuation architectures and provide a methodological baseline for the present system.

III. Methodology

A. System Architecture

The system follows a linear signal chain from image acquisition to conveyor actuation. A USB camera mounted above the conveyor belt feeds frames to a local edge computing machine running the detection and monitoring software. On a positive detection, the machine sends a short plaintext command over a raw TCP socket to an ESP32 microcontroller, which actuates a stepper motor driver to halt the belt. The architecture is:

[USB Camera] → [Edge Machine: Streamlit App + YOLOv8n Inference] → [TCP Socket / Wi-Fi] → [ESP32 Microcontroller] → [Stepper Motor Driver] → [Conveyor Belt STOP]

Local edge inference is used throughout to eliminate cloud latency and maintain operation during internet outages. Raw TCP sockets are used for ESP32 communication in preference to HTTP, reducing connection overhead and simplifying the embedded firmware.

B. Hardware Components

The camera is a USB camera interfaced via OpenCV's VideoCapture. Its resolution, frame rate, and DirectShow/AVFoundation backend are configured at startup according to the host operating system; the capture buffer size is explicitly set to one frame to minimize lag and ensure inference always operates on the most recent image. The edge machine runs Python 3 with the Ultralytics, OpenCV, NumPy, Pandas, Streamlit, and Pillow libraries. The ESP32 microcontroller hosts a TCP server on the local Wi-Fi network and drives a stepper motor controller via GPIO signals. A dedicated inspection light - controlled by the ESP32 through light_n (on) and light_f (off) commands - provides stable illumination independently of factory ambient conditions.

C. Dataset Creation and Labeling

A custom dataset was compiled by capturing images directly from the operational conveyor setup under real factory lighting conditions. The dataset spans two categories: clean tea frames (negative samples) and frames containing foreign particles (positive samples). Contaminant types introduced include cotton threads, synthetic fibers, plastic fragments, stone chips, and insect specimens. All images were annotated using LabelImg [12] in YOLO label format, with bounding boxes drawn tightly around each contaminant instance. The annotated dataset was divided into training (70%), validation (20%), and test (10%) subsets. Augmentation techniques - horizontal and vertical flips, brightness jitter, and random cropping - were applied during training to improve generalization across positional and illumination variation.

D. Model Training

Faster R-CNN was implemented using Torchvision with a ResNet-50 ImageNet-pretrained backbone, fine-tuned using SGD with momentum. YOLOv11 and YOLOv8n were both trained for 100 epochs using the Ultralytics library with default Adam optimizer settings. All three models were trained on identical dataset splits to ensure comparability of evaluation results.

IV. System Design and Implementation

A. Detection Pipeline

The software pipeline is implemented in Python and organized as a Streamlit application. The YOLOv8n model weights are loaded once at startup using @st.cache_esource and reused across all inference cycles. The camera is managed through an ensure_camera() function that initializes VideoCapture with the specified resolution, frame rate, and backend, and reuses the same capture object across cycles to avoid reconnection overhead.

Each inference cycle calls read_latest_frame(), which flushes a configurable number of buffered frames using cap.grab() before calling cap.read(). This ensures the model always operates on the most current frame rather than a stale buffer entry - particularly important at higher belt speeds where a delay of even a few frames corresponds to meaningful spatial displacement on the belt. Inference is performed by run_yolo_on_bgr_frame(), which calls model.predict() with the configured confidence threshold, image size, and compute device, extracts bounding boxes and confidence scores from

the result, and returns both the annotated frame and a list of per-detection records for display.

Detection decisions are governed by a configurable confidence threshold τ. For each candidate bounding box, the model outputs a combined confidence score. The detection decision rule applied in run_yolo_on_bgr_frame() is:

$$\hat{y} = 1 \quad \text{if confidence} \geq \tau, \quad \text{else } 0 \quad (1)$$

where τ is the configurable confidence threshold (default τ = 0.15, range 0.05–0.95)

B. Burst Inference and Auto-Stop Modes

To reduce the probability of missing a genuine contaminant due to a single-frame gap, the system implements burst inference: a configurable number of frames B is captured and processed sequentially within each monitoring cycle. This is particularly effective for small or fast-moving particles that may appear clearly in one frame but be at the edge or partially occluded in adjacent ones.

In Any Detection mode, a cycle is flagged as positive if at least one frame in the burst returns a detection. The cycle detection rule, implemented directly as the cycle_has_detection flag in the live inference loop, is:

$$D_{cycle} = D_1 \vee D_2 \vee \ldots \vee D_B \quad (2)$$

where $D_i = 1$ if frame i contains a detection, and $B \in \{1, 2, 3, 4, 5\}$ is the burst frame count

In Consecutive Hit Frames mode, the system requires N successive positive cycles before issuing a STOP command, reducing susceptibility to isolated transient false positives. The stop condition, implemented as the consecutive_hits >= stop_frames check, is:

$$\text{STOP} = 1 \quad \text{if } \Sigma D_k \geq N, \quad \text{for } k = n-N+1 \text{ to } n \quad (3)$$

where N is the required consecutive hit count (stop_frames, range 1–5) and n is the current cycle index

A latch flag (auto_stop_latched) prevents redundant stop commands from being re-sent while the belt is already halted, and is cleared only when an operator issues a resume command.

C. TCP Communication with the ESP32

Actuation commands are sent via send_esp32_command(), which opens a TCP connection to the ESP32's IP and port using socket.create_connection(), transmits the command as a UTF-8 newline-terminated string, and optionally reads a response within a configurable timeout. The ESP32 firmware interprets four commands: run (start conveyor), stop (halt conveyor), light_n (light on), and light_f (light off). The use of raw TCP rather than HTTP eliminates header parsing overhead on the embedded device and reduces per-command round-trip time.

When Pulse RUN compatibility mode is enabled, the run command must be periodically re-sent to keep the conveyor moving. The re-send condition, implemented in the run_live_cycle() function, is:

$$\text{send\_run} = 1 \quad \text{if } (t_{now} - t_{last\_run}) \geq \Delta t_{pulse} \quad (4)$$

where Δt_{pulse} is the configurable pulse interval in seconds (default 1.0 s, range 0.2–10.0 s)

A dry-run mode is provided for testing: when active, no socket connection is opened and the command is logged locally as a DRY-RUN event, enabling full pipeline validation without physical hardware.

D. Monitoring Dashboard

The Streamlit dashboard is divided into three tabs. The Live Monitor tab displays the real-time annotated camera feed, KPI widgets for belt state, cumulative stop events, and last detection count, and controls for starting/stopping monitoring, resuming the belt, and adjusting runtime parameters including confidence threshold, image size, burst count, frame flush count, auto-stop mode, and refresh interval. The Single Image Inspection tab accepts uploaded images for on-demand analysis, displaying input and annotated output side by side with a download option. The Manual Line Control tab provides direct buttons for Conveyor RUN, Emergency STOP, Light ON, and Light OFF, together with a timestamped event log and an inspection trend chart of detection counts over the last 30 cycles.

V. Results and Evaluation

A. Model Performance Comparison

All three models were evaluated on the held-out test set. Table 1 reports mAP@0.5, mAP@0.5:0.95, precision, recall, and F1-score for each model. These are the actual values obtained from the evaluation runs on the project dataset.

Table 1: Comparative Performance Metrics on the Foreign Particle Test Set

Model	mAP@0.5	mAP@0.5:0.95	Precision	Recall	F1-Score
Faster R-CNN	74.15%	33.25%	2.48%	93.75%	4.83%
YOLOv11	78.08%	48.54%	100.00%	52.73%	69.05%
YOLOv8n (Selected)	**78.85%**	**48.11%**	**99.18%**	**59.38%**	**74.28%**

Highlighted row indicates the model selected for deployment. All metrics computed on the held-out test split

B. Analysis of Model Behavior

The results expose strikingly different failure modes across the three models. Faster R-CNN achieved the highest recall (93.75%) but at a precision of only 2.48%, yielding an F1-score of 4.83%. This pattern is characteristic of a model that over-generates candidate detections: the RPN proposes a large number of regions, and the classifier accepts the overwhelming majority of them regardless of confidence. In operational terms, a precision of 2.48% would mean the conveyor is falsely stopped approximately 40 times for every

genuine contaminant - rendering the system unusable. Its mAP@0.5 of 74.15% appears more favorable because that metric averages over the full precision-recall curve; at any deployable operating point, the model performs poorly.

YOLOv11 exhibited the opposite behavior: a perfect precision of 100% with a recall of only 52.73%, producing an F1-score of 69.05%. Every detection it makes is correct, but it misses nearly half of all actual contaminants. In a food safety context this is equally problematic - undetected particles reaching consumers represent the primary risk the system exists to prevent.

YOLOv8n strikes the most operationally viable balance. Its precision of 99.18% is nearly identical to YOLOv11's, meaning false alarms remain extremely rare. Its recall of 59.38%, while not high in absolute terms, represents a meaningful improvement over YOLOv11 and a dramatically better operating point than Faster R-CNN. The F1-score of 74.28% and mAP@0.5 of 78.85% are the highest among the three models. Its compact nano architecture additionally delivers the fastest inference speed of the three, making it the most suitable candidate for real-time edge deployment.

C. Recall Limitation

The important observation from Table 1 is that all three models exhibit limited recall on this dataset, with the best (Faster R-CNN, at 93.75%) achieving it only at the cost of catastrophic precision degradation. YOLOv8n's 59.38% recall means the deployed system will miss approximately four in ten foreign particle instances under the conditions represented in the test set. This is primarily a data limitation: the training set contains a relatively small number of annotated contaminant instances spread across several morphologically diverse classes (threads, plastic, insects, stones), giving the model insufficient exposure to each contaminant type to generalize reliably. Targeted dataset expansion - particularly adding more contaminant samples and applying hard-negative mining is the most direct path to improving recall in future iterations.

VI. Discussion

A. Advantages of the System

The system operates continuously without fatigue, providing consistent inspection coverage across full production shifts. The edge-computing architecture eliminates reliance on internet connectivity, ensuring operation during network outages and avoiding the latency variability of cloud inference. The use of raw TCP sockets for ESP32 communication reduces per-command overhead relative to HTTP, contributing to lower actuation latency. The dry-run mode enables full software testing without hardware, reducing development risk. Integrated lighting control via the ESP32 allows the inspection light to be automatically switched on when monitoring starts and off when it stops, decoupling illumination consistency from factory ambient conditions and improving detection stability.

The Streamlit dashboard requires no dedicated industrial terminal and is accessible from any browser on the local network. The burst inference mechanism directly addresses a practical gap in single-frame approaches: small particles that might be missed in one frame due to motion, partial occlusion, or position near the image boundary are more likely to appear clearly in at least one frame within a burst, improving effective detection coverage without a model retraining effort.

B. Limitations

The 59.38% recall of the deployed model is the most significant operational limitation. While the near-perfect precision ensures that the system rarely causes unnecessary stoppages, a miss rate of approximately 40% means the system should currently be treated as a supplementary layer alongside - rather than a replacement for - human inspection. The recall limitation is expected to diminish substantially with a larger and more diverse annotated dataset.

Factory lighting variability poses a secondary challenge. Although the dedicated inspection light mitigates this, sudden ambient changes - a door opening, a nearby machine activating - can transiently alter image characteristics in ways the model was not trained on. A fully enclosed imaging chamber with controlled illumination would provide the most robust solution. Camera vibration transmitted through the conveyor frame can also introduce motion blur at higher belt speeds; a vibration-dampened mounting bracket is recommended for permanent installation.

C. Industrial Applicability

The architecture is modular and straightforwardly adaptable to other granular food processing lines such as spice packaging, grain milling, and dried fruit sorting. The ESP32 command vocabulary could be extended to control pneumatic ejectors or diverter gates, enabling targeted removal of contaminated portions without halting the entire belt - a meaningful throughput advantage in high-volume operations. The total hardware cost of the system - a USB camera, a standard PC, an ESP32 board, and a stepper motor driver - is modest relative to proprietary optical inspection equipment, making it financially accessible to small and medium manufacturers.

VII. Conclusion and Future Work

This paper has presented a complete IoT-based foreign particle detection system for tea manufacturing, integrating real-time YOLOv8n inference with TCP-based ESP32 conveyor control and a Streamlit monitoring dashboard. A comparative evaluation of Faster R-CNN, YOLOv11, and YOLOv8n on a custom annotated dataset revealed distinct and complementary failure modes: Faster R-CNN's precision collapsed to 2.48%, making it operationally infeasible; YOLOv11 achieved perfect precision at the cost of a 52.73% recall; and YOLOv8n provided the best balance, with 99.18%

precision, 59.38% recall, and a 74.28% F1-score. YOLOv8n was accordingly selected as the sole deployed model.

The principal contributions of this work are: (1) a custom annotated tea conveyor image dataset reflecting realistic industrial contamination; (2) a comparative evaluation that exposes the concrete failure modes of three leading detection architectures on this problem; and (3) a fully integrated, lowcost inspection and actuation system with burst inference, dual auto-stop modes, TCP-based ESP32 communication, and integrated lighting control.

Future work will focus on expanding the annotated dataset - particularly increasing per-class contaminant sample counts and applying hard-negative mining - to improve recall toward a level sufficient for standalone deployment. Deployment of the model on an edge AI accelerator such as the NVIDIA Jetson Nano or Google Coral USB Accelerator would reduce hardware cost and energy consumption. Model quantization and pruning present a longer-term pathway toward on-device inference on the ESP32 itself. Multi-class contaminant classification would enable risk-tiered responses - advisory alerts for minor contamination versus immediate halts for high-risk objects - moving the system toward structured quality risk management.

REFERENCES

[1] Food and Agriculture Organization of the United Nations (FAO), ”Tea Market Report 2023,” FAO Commodities and Trade Division, Rome, Italy, 2023.

[2] T. Brosnan and D.-W. Sun, ”Improving quality inspection of food products by computer vision - a review,” Journal of Food Engineering, vol. 61, no. 1, pp. 3-16, Jan. 2004.

[3] P. Jiang, Y. Chen, B. Liu, D. He, and C. Liang, ”Real-time detection of apple leaf diseases using deep learning approach based on improved convolutional neural networks,” IEEE Access, vol. 7, pp. 59069-59080, 2019.

[4] Z. Chen, T. Niu, Y. Xiao, and H. Ying, ”Hyperspectral imaging technology for quality and safety evaluation of tea,” Food and Bioprocess Technology, vol. 14, pp. 2033-2052, 2021.

[5] R. Girshick, J. Donahue, T. Darrell, and J. Malik, ”Rich feature hierarchies for accurate object detection and semantic segmentation,” in Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), Columbus, OH, USA, 2014, pp. 580-587.

[6] S. Ren, K. He, R. Girshick, and J. Sun, ”Faster R-CNN: Towards real-time object detection with region proposal networks,” IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 39, no. 6, pp. 1137-1149, Jun. 2017.

[7] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, ”You only look once: Unified, real-time object detection,” in Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), Las Vegas, NV, USA, 2016, pp. 779-788.

[8] Ultralytics, ”YOLOv8: A New Frontier in Object Detection,” GitHub Repository, 2023. [Online]. Available: https://github.com/ultralytics/ultralytics.

[9] Ultralytics, ”YOLOv11: Next-Generation Object Detection,” Ultralytics Documentation, 2024. [Online]. Available: https://docs.ultralytics.com.

[10] A. Kumar and S. Patel, ”ESP32-based IoT quality inspection system for automated manufacturing lines,” International Journal of Advanced Manufacturing Technology, vol. 118, pp. 3121-3132, 2022.

[11] R. Fernandez, L. Torres, and M. Castillo, ”Automated defect detection in bottling plants using YOLO-based machine vision and PLC integration,” Computers and Electronics in Agriculture, vol. 199, p. 107155, 2022.

[12] T. Lin, ”LabelImg: Graphical Image Annotation Tool,” GitHub Repository. [Online]. Available: https://github.com/tzutalin/labelImg. Accessed: Jan. 2025.